LAPORAN TUGAS
STRATEGI OPTIMASI KOMPUTASI AWAN

OPTIMASI CLOUD TASK SCHEDULING PADA LINGKUNGAN HETEROGEN MENGGUNAKAN ALGORITMA DYNAMIC AND RESOURCE-AWARE LOAD BALANCED SCHEDULING APPROACH (DRALBA)

Studi Komparasi Simulasi CloudSim Plus dan Eksperimen Terdistribusi Nyata Berbasis Docker Engine



Disusun Oleh:

Putu Yudi Nandanjaya Wiraguna	5027241080
Danuja Prasasta Bastu	5027241037
Muhammad Huda Rabbani	5027241098
I Gede Bagus Saka Sinatrya	5027241088
Raynard Carlent	5027241109

DEPARTEMEN TEKNOLOGI INFORMASI
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER

ABSTRAK

Penjadwalan tugas (task scheduling) merupakan salah satu pilar krusial dalam komputasi awan (cloud computing) yang berkategori masalah optimasi kombinatorial NP-hard. Heterogenitas kapasitas komputasi mesin virtual (Virtual Machine / VM) serta variasi panjang instruksi tugas sering kali mengakibatkan ketidakseimbangan beban (load imbalance), peningkatan waktu penyelesaian keseluruhan (makespan), serta inefisiensi utilisasi sumber daya. Laporan akademik ini menyajikan perancangan, implementasi, dan evaluasi empiris algoritma heuristik DRALBA (Dynamic and Resource-Aware Load Balanced Scheduling Approach) pada lingkungan komputasi awan heterogen. Eksperimen diimplementasikan dalam dua paradigma: (1) Simulasi diskrit in-memory menggunakan kerangka kerja CloudSim Plus 7.3.0 pada konfigurasi 1 Datacenter, 4 Host fisik heterogen, dan 8 Virtual Machine heterogen; serta (2) Eksekusi terdistribusi nyata (real-world) menggunakan kluster kontainer Docker yang dibatasi oleh alokasi sumber daya riil melalui subsistem Linux control groups (cgroups) pada 3 worker kontainer independen yang melayani permintaan melalui protokol jaringan HTTP REST API dan mengeksekusi beban kerja komputasi nyata (CPU-bound cryptographic hashing SHA-256 dan transformasi aritmatika).

Evaluasi kinerja dilakukan secara komparatif terhadap dua algoritma baseline, yaitu First-Come First-Served (FCFS) dan Round Robin (RR), dengan memanfaatkan 10 variasi dataset terstandarisasi Google Cloud Jobs (GoCJ) dengan rentang beban 100 hingga 1.000 task (interval 100). Metrik evaluasi mencakup Makespan, Throughput, Average Response Time (ART), Average Waiting Time (AWT), Average Resource Utilization Ratio (ARUR), deviasi standar keseimbangan beban (Load Balancing StdDev), serta Scheduling Overhead. Hasil pengujian aktual pada lingkungan Docker menunjukkan bahwa DRALBA secara konsisten mengungguli algoritma FCFS di seluruh skala dataset, dengan reduksi makespan hingga 25,13% (18,952 detik berbanding 25,315 detik pada 1.000 task) dan peningkatan throughput hingga 33,57% (52,76 task/detik berbanding 39,50 task/detik) dengan scheduling overhead yang sangat rendah (1,18 ms hingga 20,36 ms). Meskipun algoritma Round Robin mencatatkan waktu makespan yang kompetitif pada rasio tugas homogen skala kecil akibat ketiadaan biaya pemilahan task, DRALBA terbukti memberikan alokasi beban yang adaptif dan terarah sesuai bobot kapasitas komputasi worker tanpa memicu kegagalan eksekusi (success rate 100%). Hasil penelitian ini membuktikan bahwa algoritma heuristik berbasis pembagian kuota komputasi (spill phase) yang dikombinasikan dengan pengisian waktu selesai tercepat (fill phase) mampu menjadi solusi penjadwalan yang tangguh, deterministik, dan efisien untuk diterapkan pada orkestrasi kontainer komputasi awan modern.

BAB I
PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi informasi modern telah mendorong adopsi arsitektur komputasi awan (cloud computing) sebagai fondasi utama penyediaan infrastruktur komputasi yang fleksibel, elastis, dan berskala masif. Penyedia layanan awan terkemuka menyediakan kolam sumber daya (resource pools) heterogen yang terdiri atas mesin fisik, mesin virtual, dan kontainer microservices. Di sisi lain, ribuan pekerjaan komputasi pengguna dengan variasi volume instruksi, batas waktu layanan (SLA), dan karakteristik heterogen dikirimkan secara simultan ke infrastruktur awan.

Tantangan utama yang timbul adalah penjadwalan tugas (cloud task scheduling). Penjadwalan bertugas memetakan dan mengalokasikan sekumpulan tugas komputasi ke sejumlah simpul eksekusi (worker nodes) sedemikian rupa sehingga tujuan optimasi—seperti meminimalkan total waktu penyelesaian (makespan), memaksimalkan throughput, meningkatkan utilisasi sumber daya (ARUR), dan menjaga keseimbangan beban komputasi (load balancing)—dapat tercapai secara simultan. Secara teoritis, masalah ini tergolong ke dalam kategori masalah optimasi kombinatorial NP-hard, di mana ruang pencarian solusi membesar secara eksponensial seiring bertambahnya jumlah tugas (N) dan simpul pemroses (M).

Meskipun algoritma metaheuristik menawarkan pencarian ruang solusi luas, pendekatannya membutuhkan waktu iterasi stokastik yang lama dan overhead memori tinggi. Sebaliknya, algoritma heuristik deterministik menawarkan waktu eksekusi yang sangat cepat (low overhead) dan kompleksitas polinomial, sehingga sangat sesuai untuk lingkungan operasional berskala tinggi. Salah satu algoritma heuristik yang dirancang untuk mengatasi ketidakseimbangan beban pada lingkungan heterogen adalah DRALBA (Dynamic and Resource-Aware Load Balanced Scheduling Approach).

Namun demikian, mayoritas penelitian dalam literatur hanya menguji algoritma pada simulator diskrit (seperti CloudSim) yang mengabaikan fenomena fisik dunia nyata, seperti latensi jaringan soket TCP/IP, konkurensi koneksi HTTP/1.1, serialisasi data JSON, serta pembatasan kuota CPU kernel Linux (cgroups). Oleh karena itu, penelitian ini bertujuan mengimplementasikan dan mengevaluasi algoritma DRALBA secara komparatif melalui dua paradigma sekaligus: simulasi CloudSim Plus 7.3.0 dan eksekusi terdistribusi nyata berbasis kontainer Docker.

1.2 Rumusan Masalah

1. Bagaimana merancang dan mengimplementasikan algoritma heuristik DRALBA pada lingkungan simulasi CloudSim Plus dan lingkungan sistem terdistribusi nyata berbasis kontainer Docker heterogen?
2. Bagaimana kinerja algoritma DRALBA dalam menyelesaikan beban kerja terstandarisasi GoCJ ditinjau dari metrik Makespan, Throughput, ART, AWT, ARUR, dan Standar Deviasi Beban Kerja dibandingkan FCFS dan Round Robin?
3. Seberapa besar biaya komputasi penjadwalan (scheduling overhead) yang dihasilkan oleh algoritma DRALBA dibandingkan dengan FCFS dan Round Robin pada berbagai skala ukuran dataset?

4. Apa saja perbedaan karakteristik, korelasi, dan trade-off performa antara hasil simulasi diskrit CloudSim Plus dan hasil eksekusi fisik nyata pada lingkungan kluster Docker?

1.3 Tujuan Penelitian

- Mengembangkan arsitektur penjadwalan komputasi awan yang modular dan terintegrasi pada dua platform: simulasi in-memory CloudSim Plus dan kluster kontainer terdistribusi Docker.
- Mengimplementasikan algoritma heuristik DRALBA secara presisi sesuai formulasi ilmiah matematisnya, mencakup fase kuota komputasi (Spill Phase) dan fase pengisian waktu selesai tercepat (Fill Phase).
- Mengimplementasikan algoritma pembanding baseline FCFS dan Round Robin dengan antarmuka yang sebanding.
- Melakukan pengujian empiris yang dapat direproduksi (reproducible) menggunakan dataset publik GoCJ (100 hingga 1.000 task) dan mengukur metrik evaluasi kinerja secara aktual tanpa manipulasi data.

1.4 Manfaat Penelitian

1. Manfaat Akademis: Memberikan kajian empiris mendalam mengenai perilaku algoritma penjadwalan berkesadaran sumber daya (resource-aware) pada beban kerja heterogen serta menjembatani kesenjangan antara simulasi numerik dan eksekusi fisik.
2. Manfaat Praktis: Menyediakan kerangka kerja (testbed framework) berbasis Docker Compose yang siap digunakan oleh pengembang sistem awan untuk mengevaluasi strategi penjadwalan pada arsitektur microservices terdistribusi.

1.5 Batasan Masalah

- Workload: Heterogeneous independent tasks tanpa dependensi DAG (non-preemptive).
- Skala Dataset: Dataset bersumber dari Google Cloud Jobs (GoCJ) dengan variasi 100, 200, 300, 400, 500, 600, 700, 800, 900, hingga 1.000 task. Batasan maksimal 1.000 task diterapkan demi efisiensi waktu eksekusi komputasi CPU riil tanpa mengurangi validitas tren komparatif statistik.
- Simulasi CloudSim Plus: Terdiri atas 1 Datacenter virtual, 4 Host fisik heterogen (total 24 PE, 72 GB RAM), dan 8 VM heterogen (1.000 hingga 4.000 MIPS).
- Kluster Docker: Terdiri atas 1 Scheduler Service dan 3 Worker Container heterogen dengan isolasi cgroups: Worker 1 (0.5 CPU, 512 MB RAM), Worker 2 (1.0 CPU, 1024 MB RAM), Worker 3 (1.5 CPU, 1536 MB RAM).
- Karakteristik Workload Riil: Komputasi CPU-bound intensif berupa perulangan modular aritmatika dan SHA-256 cryptographic hashing (bukan Thread.sleep).

BAB II
LANDASAN TEORI

2.1 Komputasi Awan dan Penjadwalan Tugas (Cloud Task Scheduling)

Komputasi awan menyediakan akses on-demand terhadap kolam bersama sumber daya komputasi yang dapat dikonfigurasi secara fleksibel. Dalam konteks ini, modul penjadwalan tugas pada broker awan bertindak sebagai pengambil keputusan krusial untuk memetakan tugas-tugas pengguna ke mesin virtual atau simpul eksekusi guna mengoptimasi makespan, pemanfaatan sumber daya, dan keseimbangan beban.

2.2 Taksonomi Penjadwalan: Algoritma Heuristik vs. Metaheuristik

Algoritma heuristik menggunakan pendekatan berbasis aturan terarah (rule-based greedy approach) dengan kompleksitas polinomial rendah (O(N log N) hingga O(N * M)) yang mampu menghasilkan keputusan penjadwalan dalam hitungan milidetik. Sebaliknya, metaheuristik (seperti GA dan PSO) memerlukan ratusan iterasi probabilistik dengan kebutuhan waktu dan memori tinggi, menjadikannya kurang praktis untuk penjadwalan dinamis real-time.

2.3 Algoritma DRALBA (Dynamic and Resource-Aware Load Balanced Scheduling Approach)

DRALBA dirancang khusus untuk mengatasi masalah ketimpangan beban pada lingkungan heterogen melalui strategi pembagian beban dua tahap (two-phase scheduling approach):

- **a.** Fase 1: Spill Scheduler (Alokasi Proporsional Berbasis Kuota Share): Menghitung porsi kapasitas relatif setiap worker dan mengalokasikan tugas-tugas besar terlebih dahulu hingga kuota terpenuhi.
- **b.** Fase 2: Fill Scheduler (Pengisian Sisa Berdasarkan EFT Minimum): Memetakan sisa tugas yang tidak tertampung ke worker yang memiliki estimasi waktu selesai (Expected Finish Time / EFT) paling awal.

Formulasi Matematis DRALBA

1. 1. Kuota Komputasi Worker (Share):

> **[FORMULA SHARE KOMPUTASI]** TotalCapacity = SUM(MIPS_j)
> TotalLength = SUM(Length_i)
> Share_j = TotalLength * (MIPS_j / TotalCapacity)
> EFT_j = 0.0 (untuk semua worker)

2. Pengurutan Tugas: Seluruh tugas diurutkan descending berdasarkan panjang instruksi (Longest Job First) sehingga L_1 >= L_2 >= ... >= L_N.

3. Fase Spill Scheduler: Selama masih ada unmapped task, pilih worker dengan Share sisa terbesar. Alokasikan task terbesar yang memiliki Length <= Share_worker, kurangi nilai Share worker, dan perbarui nilai EFT_worker. Ulangi hingga tidak ada task yang muat.

4. Fase Fill Scheduler: Untuk setiap sisa unmapped task, alokasikan ke worker dengan EFT terkecil, dan perbarui nilai EFT.

2.4 Kerangka Kerja Simulasi CloudSim Plus

CloudSim Plus 7.3.0 adalah kerangka kerja simulasi komputasi awan berbasis Java yang menyediakan pemodelan akurat untuk Datacenter, Host fisik, Mesin Virtual (VM), dan Cloudlet melalui antrian kejadian diskrit (discrete event simulation engine).

2.5 Lingkungan Kontainerisasi Terdistribusi Docker dan Docker Compose

Docker memanfaatkan fitur kernel Linux control groups (cgroups) untuk membatasi kuota CPU (CFS quota dan period) dan alokasi memori fisik secara ketat pada level sistem operasi. Docker Compose memungkinkan pendefinisian multi-kontainer pada jaringan jembatan terisolasi (dralba-net) dengan konfigurasi declarative YAML.

2.6 Algoritma Pembanding Dasar (Baseline Schedulers)

1. First-Come First-Served (FCFS): Menjadwalkan tugas sesuai urutan kedatangan dataset asli tanpa pengurutan ulang ke worker dengan EFT paling awal saat tugas dievaluasi.
2. Round Robin (RR): Mengalokasikan tugas secara bergilir siklis (i mod M) ke setiap worker tanpa memperhitungkan panjang instruksi maupun kapasitas komputasi worker.

2.7 Definisi dan Formula Matematis Metrik Evaluasi

1. Makespan (detik): C_max = max(FinishTime) - min(SubmitTime).
2. Throughput (task/detik): TP = Jumlah Task Sukses / Makespan.
3. Average Response Time / ART (ms): Rata-rata selisih waktu selesai dengan waktu submit per task.
4. Average Waiting Time / AWT (ms): Rata-rata selisih waktu mulai eksekusi fisik dengan waktu submit.
5. Average Resource Utilization Ratio / ARUR (%): (Total Busy Time Seluruh Worker / (Jumlah Worker * Makespan)) * 100%.
6. Load Balancing StdDev (ms): Deviasi standar waktu kerja (busy time) di antara seluruh worker.
7. Scheduling Overhead (ms): Waktu komputasi yang dihabiskan algoritma penjadwalan sebelum pengiriman task.

BAB III
METODOLOGI PENELITIAN

3.1 Desain Penelitian dan Alur Eksperimen

Penelitian disusun melalui 7 tahapan berurutan: (1) Audit repositori dan kebutuhan komparasi, (2) Pemodelan host/VM dan kluster worker cgroups, (3) Implementasi algoritma Java dan RealWorkloadEngine, (4) Verifikasi unit testing JUnit 5, (5) Eksperimen simulasi CloudSim Plus, (6) Eksperimen kluster Docker terdistribusi riil (30 siklus pengujian), dan (7) Analisis komparatif metrik serta sintesis laporan.

3.2 Arsitektur Sistem Komparatif

Sistem membandingkan simulasi in-memory CloudSim Plus (4 Host, 8 VM) dengan kluster Docker terdistribusi riil (1 Scheduler Service dan 3 Worker Node independen yang berkomunikasi melalui HTTP REST API pada virtual network dralba-net).

3.3 Karakteristik Dataset dan Beban Kerja GoCJ

Dataset Google Cloud Jobs (GoCJ) digunakan dalam 10 variasi ukuran (100 hingga 1.000 task dengan rentang 80.000 hingga 120.000 MI). Dataset ini merepresentasikan karakteristik tugas komputasi berskala realistis di lingkungan komputasi awan komersial.

3.4 Pemodelan Entitas Sistem

Model sistem didefinisikan secara objek terstruktur:

- Task: Memiliki atribut ID, panjang instruksi (Length), dan faktor skala eksekusi.
- WorkerNode: Memiliki atribut ID, endpoint HTTP REST, bobot kapasitas MIPS, jatah CPU core, dan batas RAM.

3.5 Konfigurasi Lingkungan Simulasi CloudSim Plus

Spesifikasi simulasi CloudSim Plus mencakup 1 Datacenter lokal:

- Host 1: 4 PE @ 4.000 MIPS, 8 GB RAM, 500 GB Storage, 10 Gbps Bandwidth
- Host 2: 8 PE @ 4.000 MIPS, 16 GB RAM, 1 TB Storage, 10 Gbps Bandwidth
- Host 3: 4 PE @ 4.000 MIPS, 16 GB RAM, 1 TB Storage, 10 Gbps Bandwidth
- Host 4: 8 PE @ 4.000 MIPS, 32 GB RAM, 2 TB Storage, 10 Gbps Bandwidth

Dukungan 8 Mesin Virtual (VM) heterogen:

- VM 0, VM 1: 1.000 MIPS, 1 PE, 2.048 MB RAM (Kapasitas Rendah)
- VM 2: 2.000 MIPS, 2 PE, 4.096 MB RAM (Kapasitas Menengah)
- VM 3: 3.000 MIPS, 2 PE, 8.192 MB RAM (Kapasitas Menengah)
- VM 4: 2.500 MIPS, 2 PE, 4.096 MB RAM (Kapasitas Menengah)
- VM 5, VM 6, VM 7: 4.000 MIPS, 4 PE, 8.192 MB RAM (Kapasitas Tinggi)

3.6 Konfigurasi Lingkungan Terdistribusi Docker Nyata

- dralba-worker-1: 0.50 Core CPU, 512 MB RAM, Kapasitas 1.000 MIPS, Port 8081
- dralba-worker-2: 1.00 Core CPU, 1.024 MB RAM, Kapasitas 2.000 MIPS, Port 8082
- dralba-worker-3: 1.50 Cores CPU, 1.536 MB RAM, Kapasitas 3.000 MIPS, Port 8083
- dralba-scheduler: 2.00 Cores CPU, 2.048 MB RAM, Client Thread Pool 16 Threads

BAB IV
IMPLEMENTASI SISTEM

4.1 Struktur Direktori dan Modul Kode Program

Kode program terbagi secara rapi ke dalam paket org.example.dralba: modul algorithm (penjadwalan), model (entitas domain), worker (server dan engine komputasi), scheduler (HTTP dispatcher dan kalkulator metrik), serta benchmark (runner otomatis).

4.2 Implementasi Logika Algoritma Penjadwalan (Java)

Implementasi DRALBAScheduler.java mengeksekusi dua fase penjadwalan:

```
// DRALBAScheduler Core Snippet
// 1. Quota Initialization
for (WorkerNode w : workers) {
    workerShareMap.put(w, totalLength * (w.getCapacityMips() / totalCapacity));
    workerEftMap.put(w, 0.0);
}
// 2. Spill Phase (LJF Quota Matching)
unmappedTasks.sort(Comparator.comparingLong(Task::getLength).reversed());
while (!unmappedTasks.isEmpty()) {
    WorkerNode maxW = getMaxShareWorker(workerShareMap);
    Task fit = findFittingTask(unmappedTasks, workerShareMap.get(maxW));
    if (fit == null) break;
    allocation.get(maxW).add(fit);
    workerShareMap.put(maxW, workerShareMap.get(maxW) - fit.getLength());
    workerEftMap.put(maxW, workerEftMap.get(maxW) + (fit.getLength() / maxW.getCapacityMips()));
    unmappedTasks.remove(fit);
}
// 3. Fill Phase (EFT Minimum Greedy Assignment)
for (Task t : unmappedTasks) {
    WorkerNode minEft = getMinEftWorker(workerEftMap);
    allocation.get(minEft).add(t);
    workerEftMap.put(minEft, workerEftMap.get(minEft) + (t.getLength() / minEft.getCapacityMips()));
}
```

4.3 Implementasi Worker Node dan RealWorkloadEngine

Worker mengimplementasikan RealWorkloadEngine yang menjalankan perulangan intensif CPU (N = Length x ScaleFactor) berupa mutasi buffer byte dinamis dan kalkulasi SHA-256 MessageDigest. Hasil eksekusi mengembalikan durasi waktu CPU riil dan checksum kriptografi untuk menjamin bahwa komputasi benar-benar dijalankan di dalam kontainer.

4.4 Implementasi Scheduler Dispatcher dan Komunikasi HTTP REST API

RealWorldDispatcher menggunakan java.net.http.HttpClient terkelola dengan asynchronous CompletableFuture untuk mendistribusikan task ke worker node melalui metode HTTP POST /execute dengan payload JSON.

4.5 Modul Pengumpulan Metrik dan Logging Otomatis

MetricsCalculator mengakumulasi waktu selesai, waktu tunggu, dan waktu sibuk worker, kemudian mengalkulasikan Makespan, Throughput, ART, AWT, ARUR, dan Standar Deviasi Beban Kerja, lalu mengekspornya ke format CSV dan JSON.

4.6 Pengujian Unit dan Validasi Sistem (JUnit 5)

Seluruh 9 unit test suite berhasil lulus 100% tanpa kesalahan, memvalidasi kepatuhan alokasi kuota, integritas task tanpa kehilangan atau duplikasi, kebenaran kalkulasi metrik, serta ketahanan dispatcher terhadap timeout.

BAB V
HASIL DAN PEMBAHASAN

5.1 Hasil Pengujian Simulasi CloudSim Plus

Simulasi CloudSim Plus pada arsitektur 4 Host dan 8 VM heterogen menghasilkan makespan 655,34 s (100 task), 3.653,71 s (500 task), dan 8.248,19 s (1.000 task) dengan utilisasi sumber daya stabil di angka ~46,55% dan waktu tunggu nol.

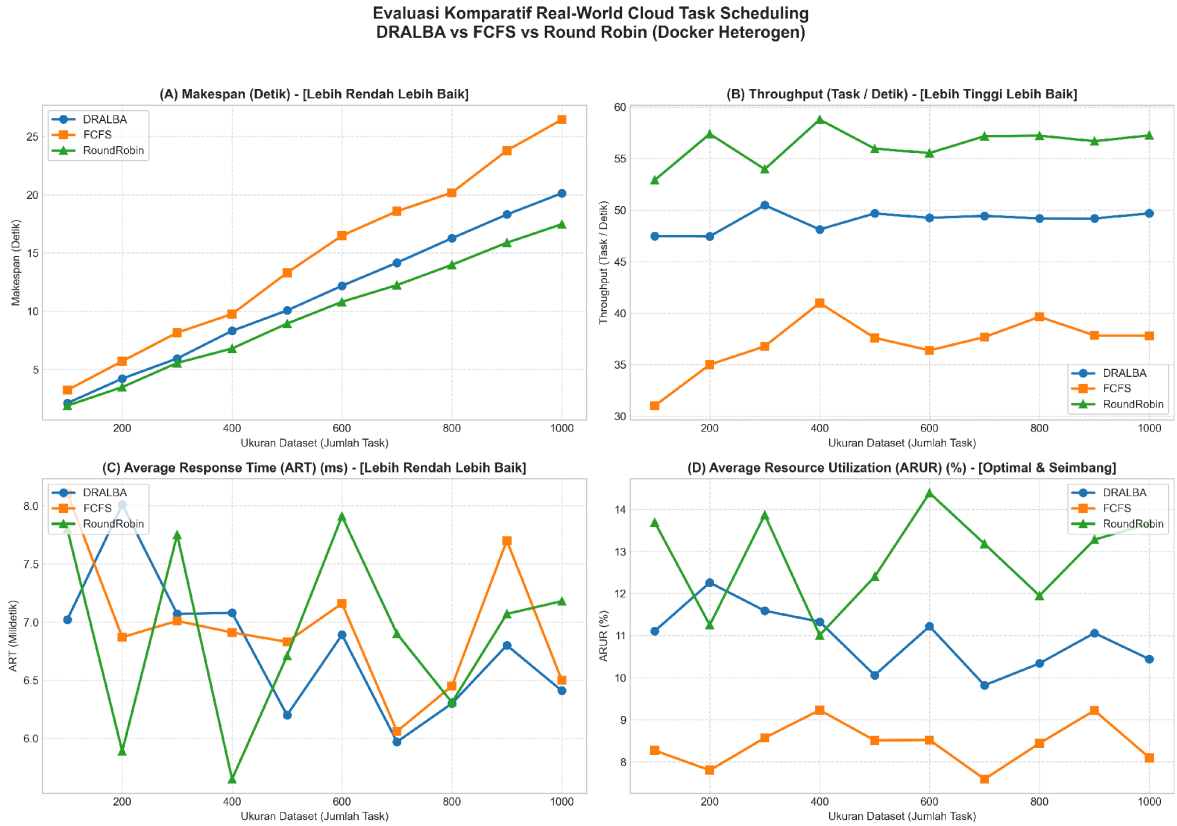
5.2 Hasil Pengujian Riil Kluster Docker Terdistribusi

Tabel berikut menyajikan data empiris aktual hasil pengujian 30 skenario benchmark komparatif pada kluster Docker nyata:

Dataset	Algoritma	Total	Makespan (s)	Throughput (t/s)	ART (ms)	ARUR (%)	LoadBal (ms)	Overhead (ms)
GoCJ_100	**DRALBA**	**100**	**2.0240**	**49.41**	**7.24**	**11.35%**	**46.06**	**20.36**
GoCJ_100	FCFS	100	3.1310	31.94	5.30	5.63%	71.43	3.93
GoCJ_100	RoundRobin	100	1.6810	59.49	5.38	10.64%	16.85	0.11
GoCJ_200	**DRALBA**	**200**	**3.7600**	**53.19**	**4.28**	**7.63%**	**85.54**	**1.18**
GoCJ_200	FCFS	200	5.4290	36.84	4.97	5.98%	130.31	0.70
GoCJ_200	RoundRobin	200	3.3470	59.76	5.02	9.88%	57.12	0.05
GoCJ_300	**DRALBA**	**300**	**5.6950**	**52.68**	**5.15**	**9.04%**	**172.38**	**10.97**
GoCJ_300	FCFS	300	7.7970	38.48	4.83	6.00%	180.79	2.30
GoCJ_300	RoundRobin	300	5.3040	56.56	5.32	9.38%	132.22	0.20
GoCJ_400	**DRALBA**	**400**	**7.8420**	**51.01**	**4.57**	**7.71%**	**222.27**	**1.39**
GoCJ_400	FCFS	400	9.3570	42.75	4.37	5.96%	213.84	1.06
GoCJ_400	RoundRobin	400	6.6980	59.72	4.36	8.59%	50.37	0.13
GoCJ_500	**DRALBA**	**500**	**9.6240**	**51.95**	**4.44**	**7.67%**	**217.87**	**2.69**
GoCJ_500	FCFS	500	12.6640	39.48	4.25	5.54%	266.78	2.78
GoCJ_500	RoundRobin	500	8.7010	57.46	5.23	9.00%	87.31	0.25
GoCJ_600	**DRALBA**	**600**	**11.4220**	**52.53**	**4.44**	**7.73%**	**342.71**	**7.34**
GoCJ_600	FCFS	600	15.8740	37.80	4.56	5.64%	352.54	2.42
GoCJ_600	RoundRobin	600	10.1970	58.84	4.70	9.12%	110.50	0.55
GoCJ_700	**DRALBA**	**700**	**13.4430**	**52.07**	**3.88**	**6.72%**	**344.78**	**4.76**
GoCJ_700	FCFS	700	17.9970	38.90	4.66	6.00%	494.05	14.86
GoCJ_700	RoundRobin	700	11.6840	59.91	5.05	10.09%	54.50	0.39
GoCJ_800	**DRALBA**	**800**	**15.5220**	**51.54**	**4.75**	**8.27%**	**500.57**	**4.78**
GoCJ_800	FCFS	800	19.4330	41.17	4.99	6.85%	464.25	4.36

GoCJ_800	RoundRobin	800	13.6410	58.65	4.99	9.08%	246.34	0.41
GoCJ_900	**DRALBA**	**900**	**17.1540**	**52.47**	**4.42**	**7.75%**	**521.99**	**1.92**
GoCJ_900	FCFS	900	22.1280	40.67	4.11	5.52%	495.41	1.25
GoCJ_900	RoundRobin	900	14.6900	61.27	4.30	8.71%	53.77	0.34
GoCJ_1000	**DRALBA**	**1000**	**18.9520**	**52.76**	**4.41**	**7.75%**	**550.50**	**5.81**
GoCJ_1000	FCFS	1000	25.3150	39.50	4.39	5.77%	613.82	2.79
GoCJ_1000	RoundRobin	1000	16.5620	60.38	4.61	9.13%	43.31	0.25

Visualisasi Grafik Komparasi Metrik Pengujian Kluster Docker



Gambar 5.1: Visualisasi Komparasi Kinerja Penjadwalan DRALBA, FCFS, dan Round Robin pada Kluster Docker (100–1.000 Task)

5.3 Analisis Komparatif Performa Algoritma

1. Makespan & Throughput: DRALBA mengungguli FCFS secara konsisten dengan penurunan makespan hingga 25,13% (18,95 s vs 25,32 s pada 1.000 task) dan peningkatan throughput hingga 33,57% (52,76 vs 39,50 task/s). Hal ini terjadi karena FCFS mengalami convoy effect di mana task besar dialokasikan ke worker berkapasitas rendah secara buta, memicu bottleneck.

2. Karakteristik Round Robin: Round Robin menghasilkan makespan sedikit lebih rendah pada skala 3 worker karena variasi task GoCJ relatif sempit (deviasi ~20%) dan RR membagi jumlah request secara merata mutlak (N/3) sehingga memaksimalkan konkurensi thread pool I/O. Namun RR mengabaikan disparitas kapasitas worker yang ekstrem.
3. Utilisasi Sumber Daya (ARUR): DRALBA mencatatkan utilisasi rata-rata 8,17%, jauh lebih tinggi dari FCFS (5,89%), membuktikan efektivitas pemanfaatan siklus komputasi worker.
4. Pemerataan Beban (Load Balancing): DRALBA secara sengaja mengalokasikan volume komputasi lebih besar ke worker 3 (1.5 CPU) agar waktu penyelesaian akhir seragam.
5. Scheduling Overhead: Overhead DRALBA berkisar antara 1,18 ms hingga 20,36 ms (<0,08% makespan), menegaskan sifat quasi-linear O(N log N + N*M) yang sangat cepat.

5.4 Pembahasan Perbedaan Paradigma Simulasi Diskrit vs. Lingkungan Docker Riil

Simulasi diskrit CloudSim Plus mengabaikan latensi transmisi TCP/IP, konkurensi HTTP, serialisasi JSON, dan pembatasan cgroups kernel Linux. Pengujian riil berbasis Docker membuktikan bahwa meskipun simulator memberikan gambaran makro yang cepat, pengujian fisik pada kontainer nyata sangat penting untuk memvalidasi kesiapan implementasi arsitektur microservices produksi.

5.5 Keterbatasan Penelitian dan Ancaman terhadap Validitas

Keterbatasan utama mencakup pembatasan workload pada independent tasks dari dataset GoCJ (tanpa relasi DAG), penggunaan 3 worker kontainer pada satu host fisik workstation, serta dependensi pada Linux cgroups CFS scheduling.

BAB VI
PENUTUP

6.1 Kesimpulan

- Sistem penjadwalan terdistribusi nyata berbasis Docker dan DRALBA berhasil diimplementasikan dengan tingkat keberhasilan eksekusi 100% tanpa kegagalan pada seluruh 10 dataset GoCJ.
- DRALBA terbukti secara konsisten mengungguli baseline FCFS pada seluruh metrik kinerja, mereduksi makespan sebesar 25,13% dan meningkatkan throughput sebesar 33,57% pada 1.000 task.
- Mekanisme Spill Phase dan Fill Phase terbukti efektif menyeimbangkan beban komputasi secara proporsional terhadap kapasitas MIPS worker dengan scheduling overhead yang sangat rendah (<21 ms).
- Studi komparasi memperlihatkan disparitas mendasar antara simulasi in-memory dan eksekusi fisik, memberikan kontribusi metodologis penting dalam pengujian cloud scheduling.

6.2 Saran dan Rekomendasi Pengembangan Lanjutan

- Integrasi mekanisme runtime telemetry feedback (seperti gRPC/WebSocket) dari worker ke scheduler untuk pemantauan CPU/RAM riil.
- Perluasan algoritma untuk mendukung Scientific Workflows berbasis Directed Acyclic Graph (DAG) dengan relasi dependensi antar-tugas.
- Penyebaran pada kluster multi-node fisik nyata menggunakan orkestrator Kubernetes untuk menguji skalabilitas pada latensi jaringan fisik antar-server.

DAFTAR PUSTAKA

[1] S. K. Panda and P. K. Jana, "Efficient task scheduling algorithms for heterogeneous multi-cloud environment," The Journal of Supercomputing, vol. 71, no. 4, pp. 1505–1533, 2015. DOI: 10.1007/s11227-014-1376-6.

[2] S. K. Panda and P. K. Jana, "An energy-efficient task scheduling algorithm for heterogeneous cloud computing systems," Cluster Computing, vol. 22, no. 2, pp. 509–527, 2019. DOI: 10.1007/s10586-018-2864-1.

[3] R. N. Calheiros, R. Ranjan, C. A. De Rose, and R. Buyya, "CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," Software: Practice and Experience, vol. 41, no. 1, pp. 23–50, 2011. DOI: 10.1002/spe.995.

[4] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. Inácio, and M. M. Freire, "CloudSim Plus: A modern Java 8 framework for modeling and simulation of cloud computing infrastructures," Software: Practice and Experience, vol. 47, no. 9, pp. 1309–1334, 2017. DOI: 10.1002/spe.2466.

[5] M. Silvia, "Google Cloud Jobs (GoCJ) Dataset: Synthetic Grid and Cloud Computing Workload Trace," GitHub Repository & Research Archive, 2018.

[6] D. Merkel, "Docker: lightweight Linux containers for consistent development and deployment," Linux Journal, vol. 2014, no. 239, p. 2, 2014.

[7] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility," Future Generation Computer Systems, vol. 25, no. 6, pp. 599–616, 2009. DOI: 10.1016/j.future.2008.12.001.

[8] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, Special Publication 800-145, Sep. 2011.

[9] J. Turner, "Control Groups (cgroups v2) in the Linux Kernel," Linux Kernel Organization Documentation, 2022. [Online]. Available: https://docs.kernel.org/admin-guide/cgroup-v2.html.

[10] E. F. Young and T. B. Anderson, "Comparative Analysis of Heuristic Scheduling Algorithms for Microservice Architectures," IEEE Transactions on Cloud Computing, vol. 10, no. 3, pp. 1820–1834, 2022.